

Assumptions and Limitations for RNA-seq on OCI Ampere A1

Date: 2026-06-23

Executive Summary

OCI Ampere A1 can run real Nextflow workflows, including an nf-core/rnaseq Kallisto pseudoalignment path, but A1 is not automatically a good fit for every RNA-seq workflow. It is strongest for highly parallel, modest-memory tasks. It is weak for memory-heavy single-process stages such as STAR genome generation and STAR alignment unless each node has enough OCPUs and RAM.

Be direct with customers:

- A1 is viable for lightweight and horizontally parallel RNA-seq workflows.
- A1 with one OCPU and 16 GB RAM per node is not a good choice for full STAR-based nf-core/rnaseq.
- A large number of tiny A1 nodes improves throughput only when every individual process fits on one node.
- For full production nf-core/rnaseq with official containers and STAR alignment, E5/E6 x86_64 shapes are usually the safer recommendation.

Tested Working Pattern

The validated A1 pattern used:

- Ampere A1 ARM64 nodes
- Slurm executor
- Apptainer container runtime
- Custom ARM64 toolbox image
- Shared /config storage
- nf-core/rnaseq 3.14.0
- Kallisto pseudoalignment
- --skip_alignment
- --skip_deseq2_qc
- strandedness=unstranded

This path completed a 1-sample smoke test and progressed through the full 24-sample workflow after avoiding the auto-strandedness branch.

Key Architectural Reality

On Ampere A1:

`1 OCPU = 1 physical Arm core = 1 schedulable Slurm CPU`

This differs from many OCI x86 shapes where:

`1 OCPU = 2 hardware threads / vCPUs`

Therefore, an A1 worker configured with one OCPU is genuinely a one-CPU Slurm worker. A cluster with 50 such nodes has 50 total task slots, but each task still gets only one CPU and the memory available on a single node.

Why 50 Small A1 Nodes Do Not Solve STAR

A cluster with:

50 nodes x 1 OCPU x 16 GB RAM

does not behave like:

1 node x 50 OCPUs x 800 GB RAM

Slurm cannot combine memory from 50 separate nodes for one STAR process. If a STAR task needs 48 GB RAM, it will fail on a 16 GB node even if 49 other nodes are idle.

Horizontal scale helps only when the largest single process fits inside one node.

Workloads That Fit A1 Well

A1 is a reasonable choice for:

- Kallisto or Salmon pseudoalignment workflows when container architecture is handled.
- FastQC and light preprocessing.
- Many independent small Nextflow tasks.
- Pipeline prototyping on ARM64.
- Cost-sensitive throughput where each task has low memory and low per-task CPU needs.
- Custom containers built and tested for ARM64.

Workloads That Are Risky on Small A1 Nodes

A1 with one OCPU and 16 GB RAM is risky for:

- STAR genome generation.
- STAR alignment against large mammalian genomes.
- Large memory-intensive R/Bioconductor steps.
- Workflows with x86_64-only containers.
- Workflows expecting AVX/x86-specific binaries.
- Nextflow pipelines that rely heavily on official Biocontainers images without ARM64 manifests.

Container Limitations

The primary blocker was not Nextflow or Slurm. It was container architecture.

Official nf-core/Biocontainers images often resolve to amd64 images. On A1, Apptainer reports errors like:

```
the image's architecture (amd64) could not run on the host's (arm64)
```

The working solution was a custom ARM64 Apptainer image with required tools installed from Ubuntu ARM64 packages and Python/R package sources.

This is production-relevant:

- Every pipeline should be tested for ARM64 container compatibility.
- A global container override can work, but the image must contain all tools needed by the allowed processes.
- Tool behavior can differ between distro packages and Biocontainers, so smoke tests are mandatory.

Singularity vs Apptainer

For A1, Apptainer is the preferred runtime in this stack.

Reasons:

- Apptainer installed cleanly on ARM64 using the Apptainer PPA.
- The attempted SingularityCE GitHub .deb path for arm64 did not exist.

- A misleading singularity package name can install an unrelated Python game, not the HPC container runtime.

Recommendation:

- Use Apptainer for ARM64.
- Use SingularityCE or Apptainer for x86_64 depending on the stack standard.

Recommendations by Scenario

Scenario 1: Customer Wants Kallisto Pseudoalignment

A1 can be a good fit.

Recommended:

```
A1 nodes with enough RAM for the largest sample-level task
Apptainer
Custom ARM64 toolbox image
Shared FSS
queueSize equal to available Slurm CPUs
```

For one-OCPU nodes:

```
process {
  cpus = 1
  memory = '14.GB'
}
```

Scenario 2: Customer Wants Full nf-core/rnaseq with STAR

Do not recommend one-OCPU A1 nodes.

Recommended options:

- Use E5/E6 x86_64 shapes with official nf-core containers.
- Use larger A1 shapes with multiple OCPUs and significantly more memory per node.
- Validate STAR memory requirements with the exact genome and annotation.

For mouse or human STAR alignment, start with nodes sized closer to:

```
8-16 OCPUs
64-128 GB RAM
```

Actual sizing depends on genome, read depth, STAR index reuse, and whether genome generation is part of the run.

Scenario 3: Customer Has 50 A1 Nodes with 1 OCPU and 16 GB RAM

Good for high-throughput Kallisto-style sample fan-out.

Not good for memory-heavy single-node stages.

Use:

```
executor {
  queueSize = 50
}

process {
  cpus = 1
  memory = '14.GB'
}
```

But do not promise full STAR alignment unless it is proven to fit within 16 GB per task.

Scenario 4: Customer Requires Official nf-core Containers

Prefer x86_64 E5/E6 shapes.

Using A1 will likely require:

- ARM64 container rebuilds.

- Tool substitution.
- Pipeline-specific testing.
- Possible patching of nf-core process assumptions.

Production Hardening Checklist

Before calling an A1 RNA-seq deployment production-ready:

1. Confirm all compute nodes have Apptainer.
2. Confirm shared storage is mounted identically on login and compute nodes.
3. Set NXF_HOME to shared storage.
4. Set Apptainer cache to shared storage or a sufficiently large node-local path.
5. Build and version the ARM64 toolbox image.
6. Store the .def file in Git.
7. Pin nf-core/rnaseq version.
8. Use a full samplesheet with explicit strandedness.
9. Run a 1-sample smoke test.
10. Run a full test dataset.
11. Capture elapsed time, CPU hours, failed retries, and node utilization.
12. Verify output matrices, MultiQC, and logs.

Bold Guidance

If the customer wants low-friction, official-container, full STAR-based nf-core/rnaseq, recommend OCI E5/E6 x86_64.

If the customer wants cost-conscious ARM64 throughput and is comfortable with custom ARM64 containers, A1 is viable for pseudoalignment and other lightweight Nextflow workloads.

If the customer asks whether many tiny A1 nodes can replace larger memory nodes for STAR, the realistic answer is no. Many small nodes improve parallelism, not per-task memory.